

Formación - Series Temporales

Transformers



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Agenda

- Introducción y breve repaso
- Attention
- Transformers
- Transformers para series temporales



Introducción



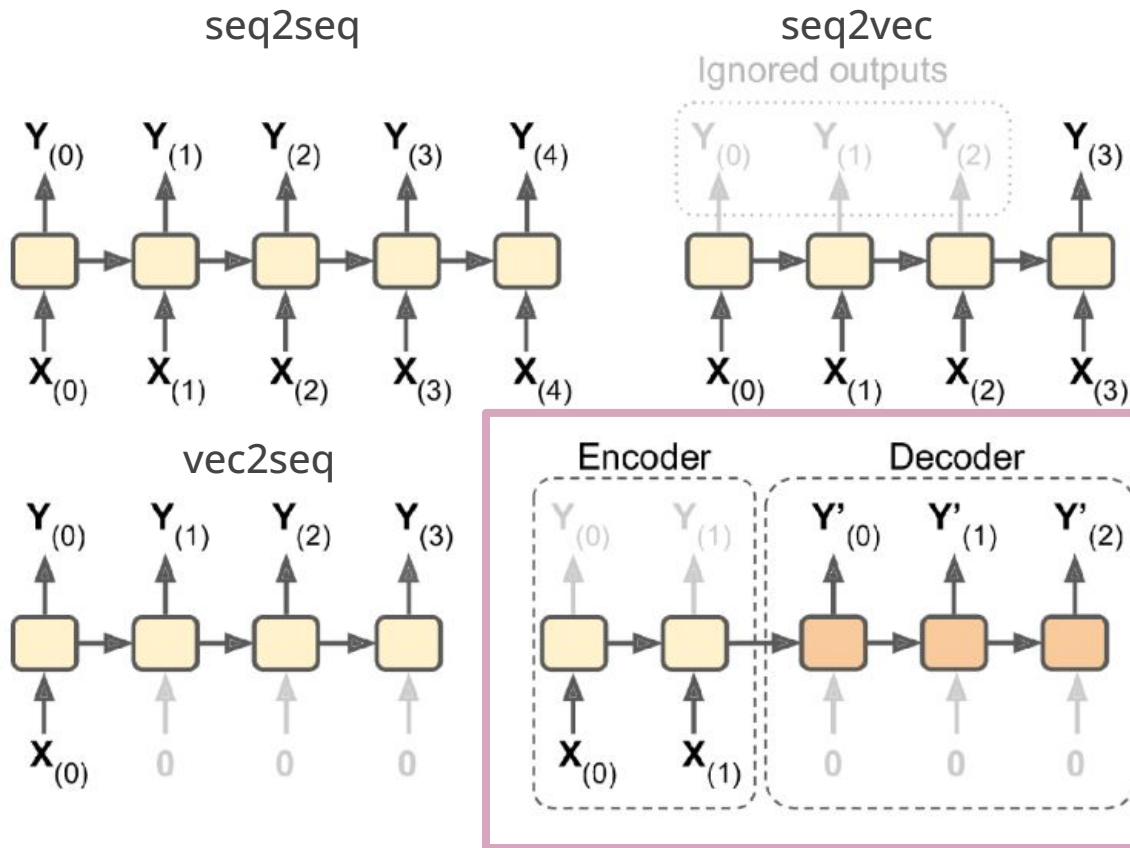
FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

REPASO: Redes recurrentes (RNN)

Diferentes configuraciones

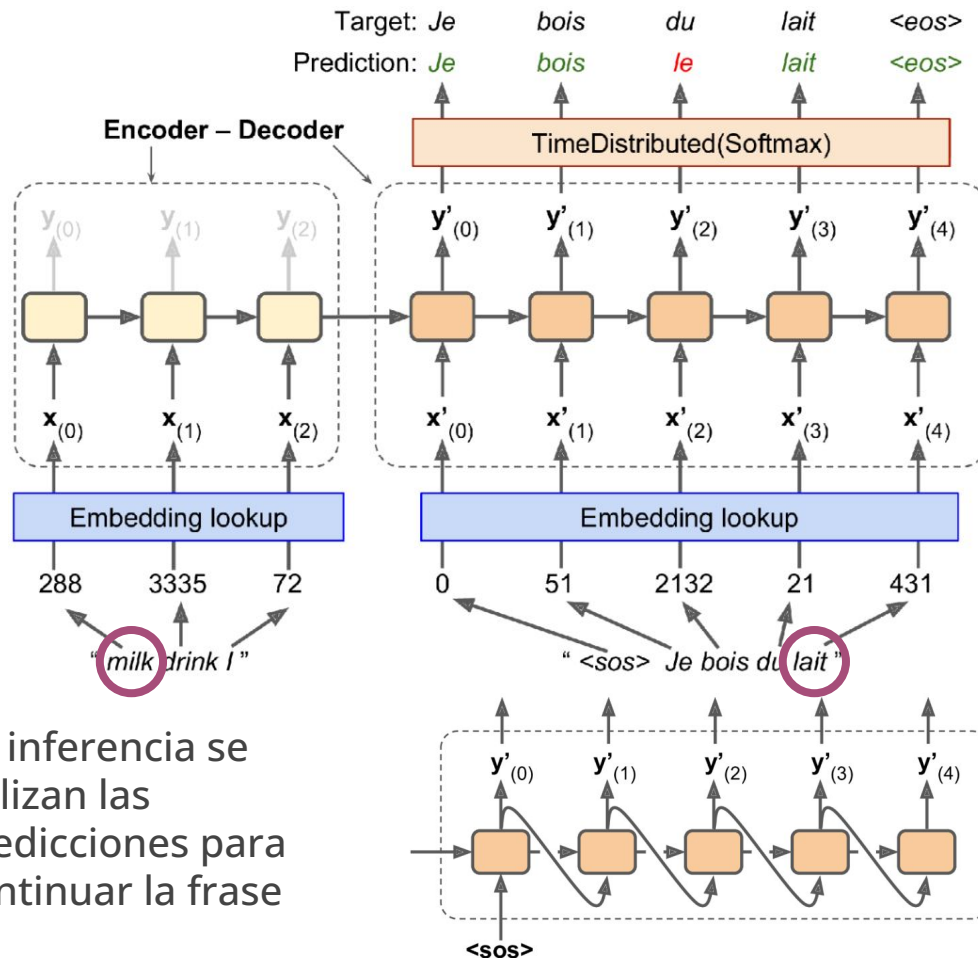


Ejemplos:

- seq2seq: predicción de una semana.
- seq2vec: predicción del siguiente valor.
- vec2seq: descripción de imágenes.
- encoder-decoder: traducción de texto.

Traductor con RNN

- Texto a números: se le asocia un ID a cada palabra y luego un vector (king - man + woman = queen).
- Encoder: procesa cada palabra de la frase a traducir de manera secuencial.
- Decoder: comienza la traducción a partir del último estado del encoder (contexto), y *start-of-sequence* < sos > . De manera secuencial devuelve las palabras más probable para cada instante.



Attention



FACULTAD DE
INGENIERÍA

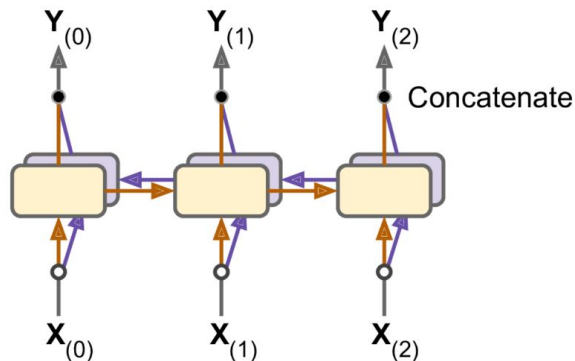
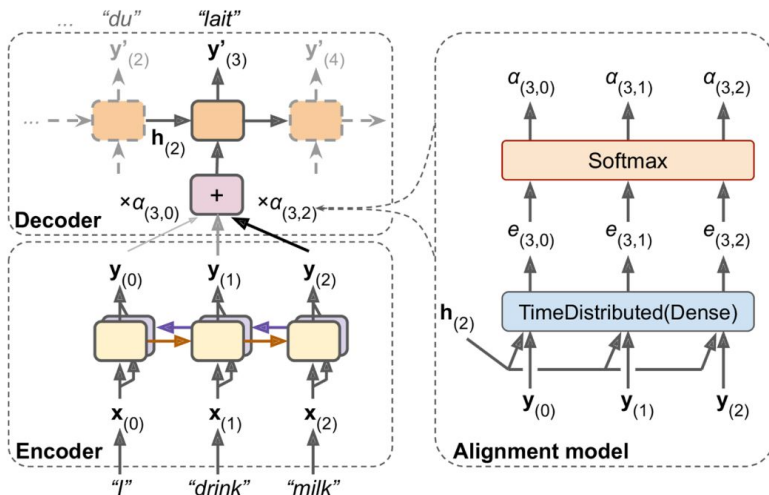


UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Traductor con RNN: Attention.

RNN Bidireccionales:

- "The queen of UK", "the queen of my hearts", y "the queen bee". Para codificar "queen" necesito información posterior a la palabra.



Attention:

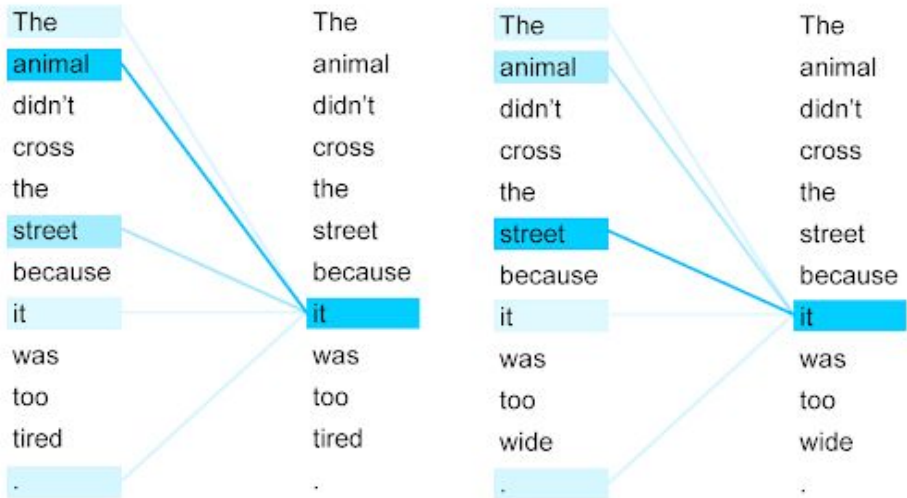
- El encoder devuelve el contexto de todos los instantes.
- Capa de **atención** (Alignment model): asocia un peso a cada vector de contexto.
- En el decoder se utiliza la suma ponderada de los contextos para estimar la palabra siguiente.

Self-Attention

- Al momento de codificar cada palabra, el contexto más relevante no tiene porqué ser el más cercano.
- "*It*" hace referencia a "animal" o "street"? Fácil para un humano pero complejo para una máquina.

Self-Attention:

- Introducido por Transformers.
- Codificación de cada palabra con el contexto más relevante en toda la oración.



Transformers



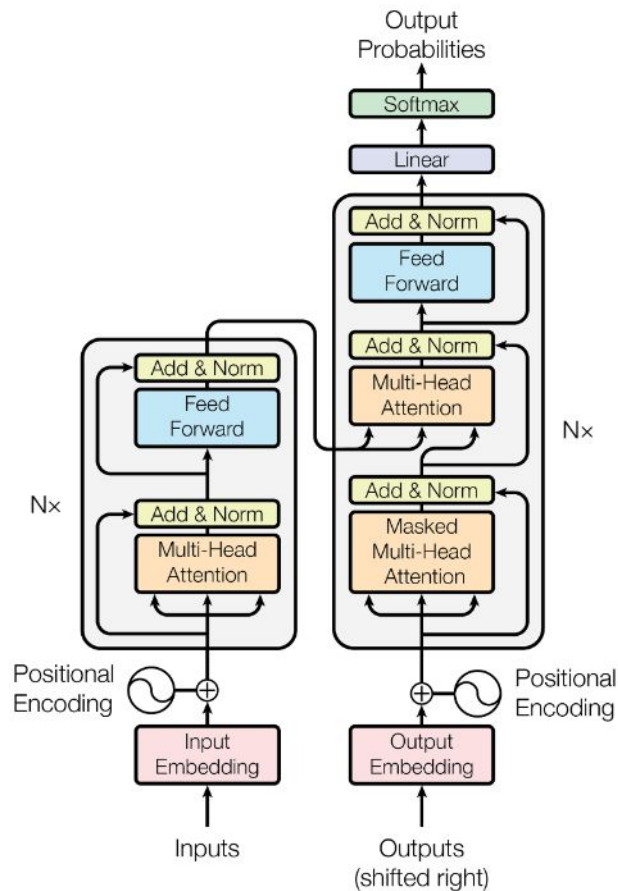
FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

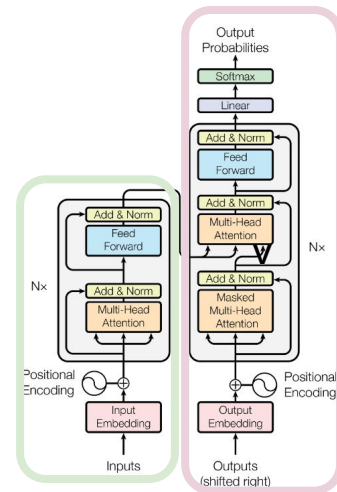
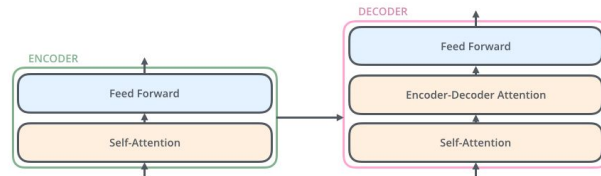
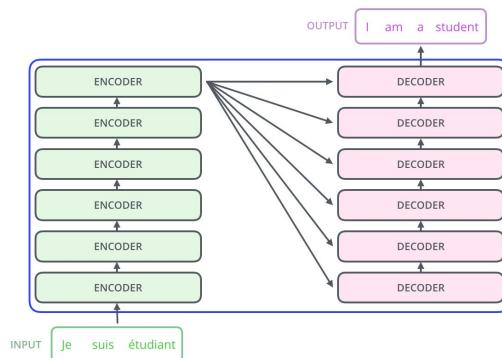
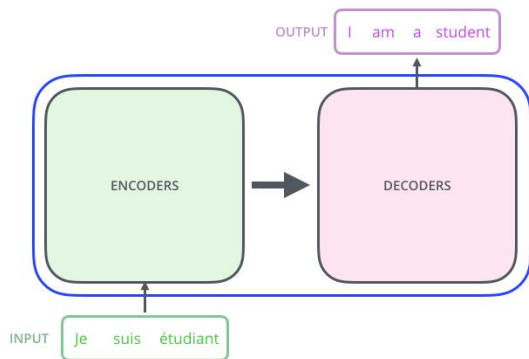
Transformers

- Superó ampliamente todo los traductores de máquina que se conocían en su momento.
- Capas totalmente paralelizables -> Más rápido de entrenar.
- Es una arquitectura clave para que la IA tenga los resultados que hoy conocemos.
- Es complejo de entender. Veamos sus partes principales.



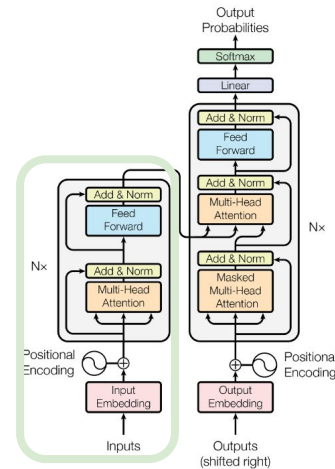
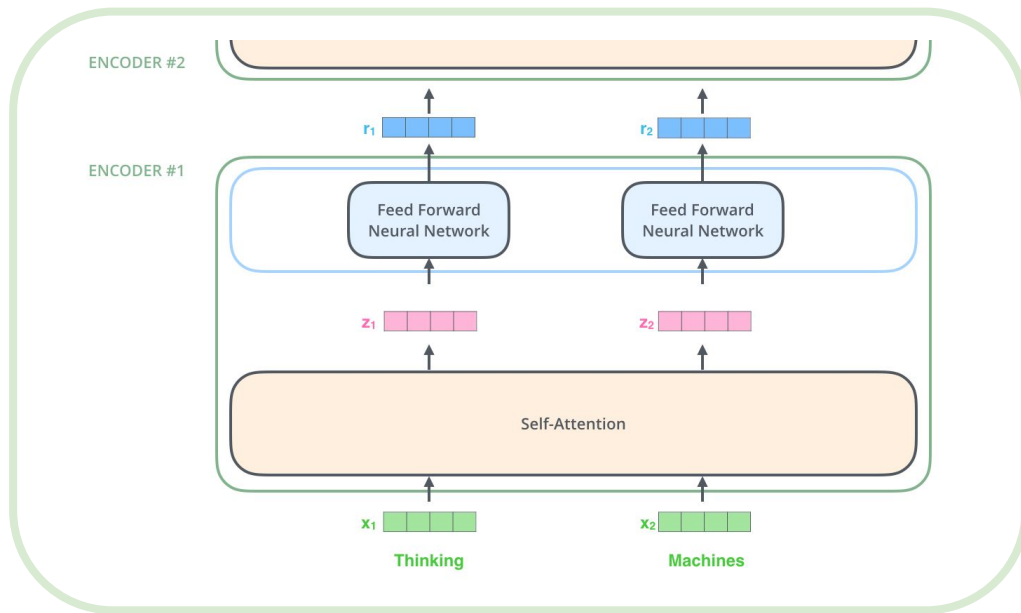
Transformers: una mirada de alto nivel.

- Es una arquitectura encoder-decoder.
- Tanto el encoder como el decoder son un stack de bloques idénticos.
- Cada bloque aplica Self-Attention.



Transformers: una mirada de alto nivel.

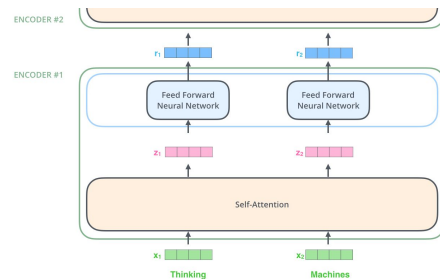
- Encoding:



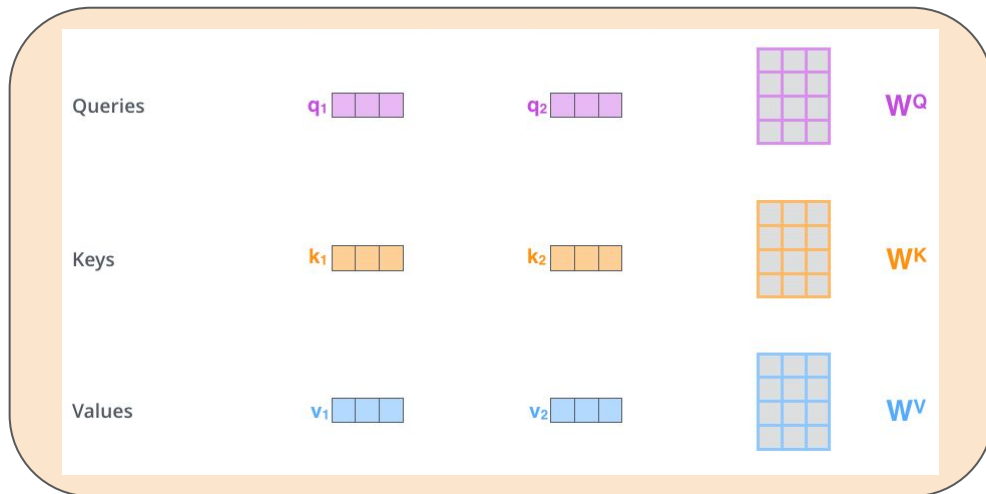
Transformers: Self-Attention en detalle

Primer paso:

- A partir de los Embedding y las matrices (aprendibles) correspondientes se crean los vectores: **Queries**, **Keys**, y **Values**.



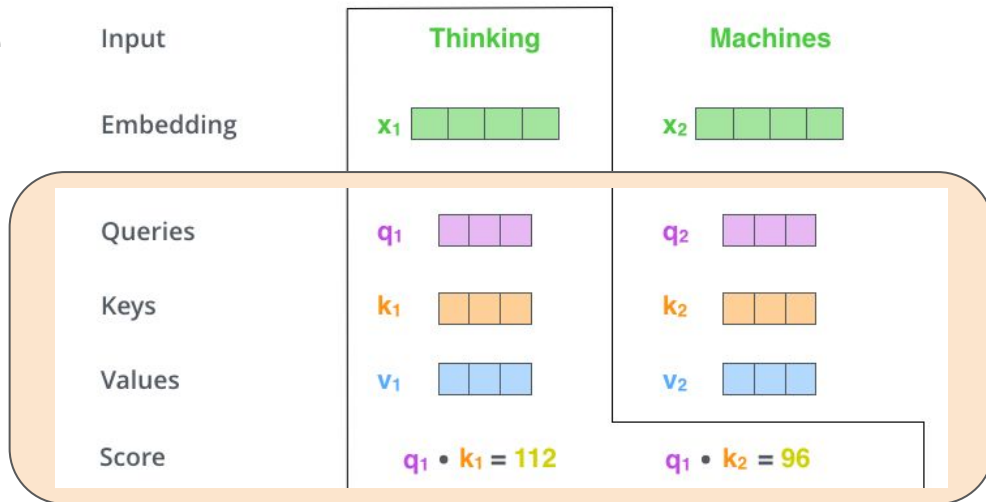
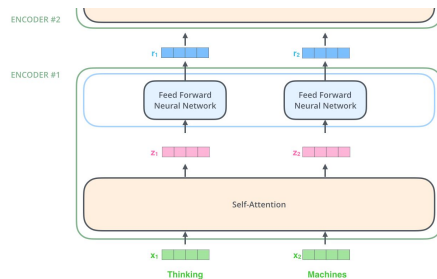
Input
Thinking Machines
Embedding x_1 x_2



Transformers: Self-Attention en detalle

Segundo paso:

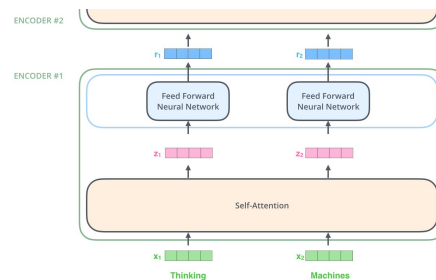
- Calcular el score: determinará cuánta atención debemos poner en el resto de las palabras de la oración.
- Los score se calcula a partir del producto punto a punto del Querie de la posición en cuestión (**q1**), y los keys de toda la entrada (**k1**, **k2**).



Transformers: Self-Attention en detalle

Tercer y cuarto paso:

- Dividir por la raíz de la dimensión de las Keys. Ayuda a estabilizar los gradientes.
- Se normaliza el score con Softmax y quedan todos los score positivos, y suman 1.
- Esta puntuación de Softmax determinará cuánto representa cada palabra de la entrada en esta posición.

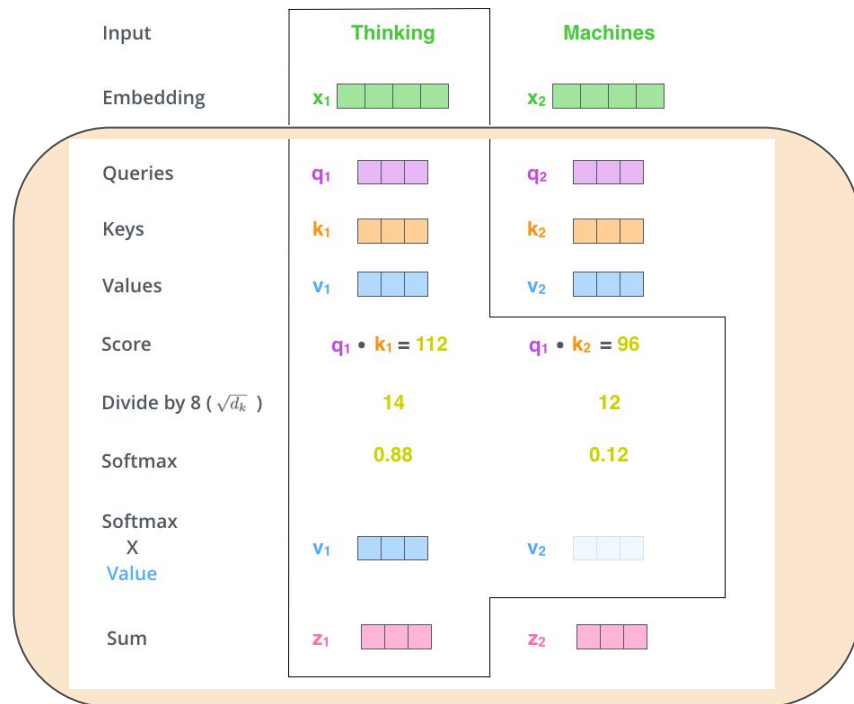
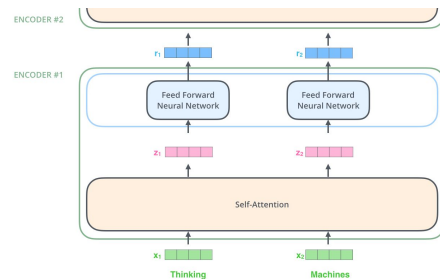
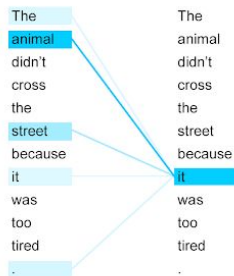


Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by $8 (\sqrt{d_k})$	14	12
Softmax	0.88	0.12

Transformers: Self-Attention en detalle

Quinto y sexto paso:

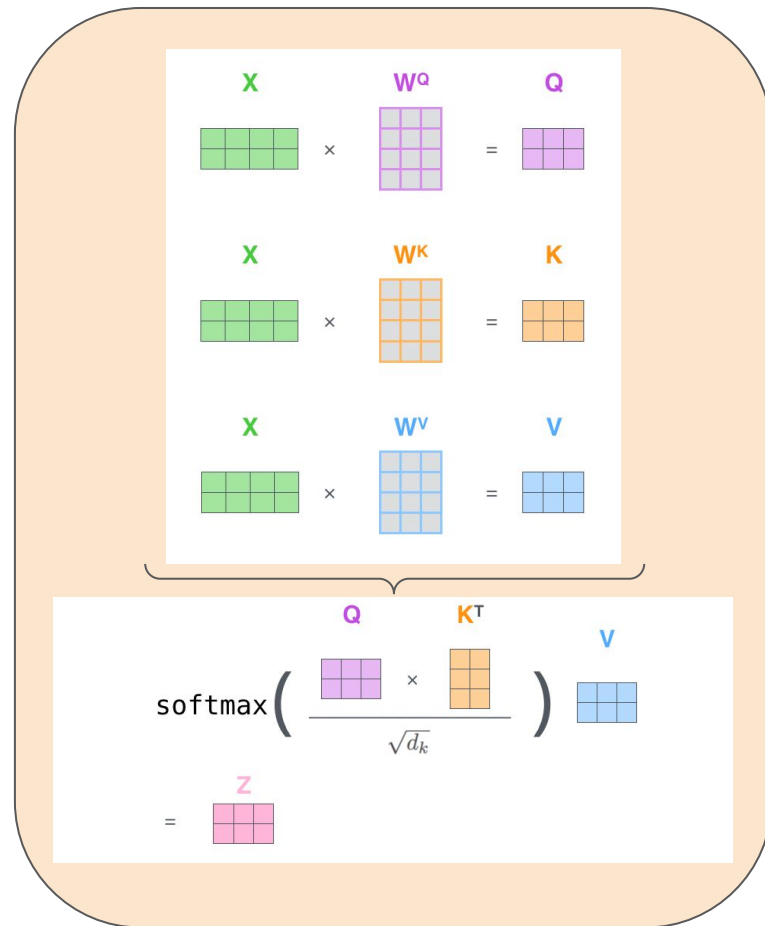
- Multiplicar cada vector Value por el score softmax.
- Sumarlos a todos, y así obtener la salida del self-attention en esa posición: **z1**.
- La intuición es mantener los Values de las palabras en las que queremos que se preste atención, y atenuar el resto.



Transformers: Self-Attention en detalle

Calculo matricial:

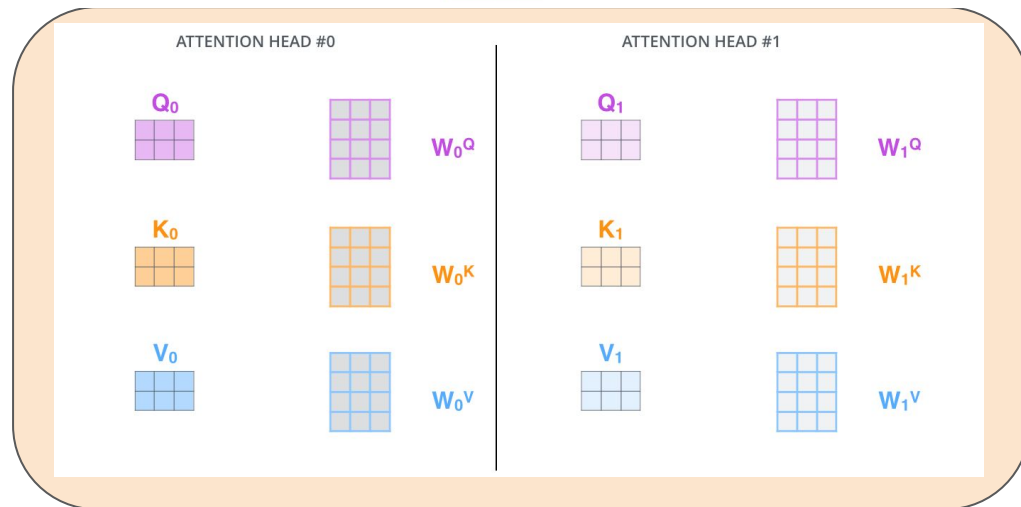
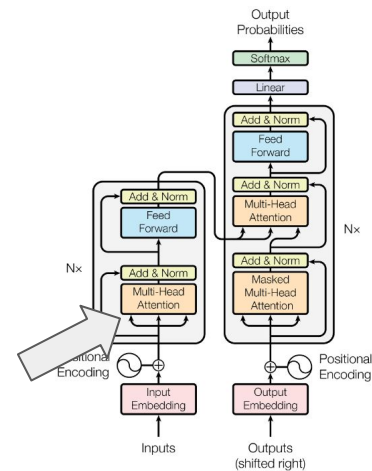
- Paraleliza los cálculos y acelera los procesos.
- Se puede ver que el largo de las secuencias no altera la cantidad de parámetros entrenables.



Transformers: Multi-headed attention

- Los pesos se inicializan de manera aleatoria por lo que cada Head proyecta la entrada en un subespacio distinto.
- Permite capturar distintos tipos de dependencias. Ej.:
 - Head #0: el significado.
 - Head #1: el tiempo verbal.

Thinking
Machines



Transformers: Multi-headed attention

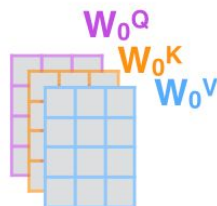
1) This is our input sentence*

Thinking
Machines

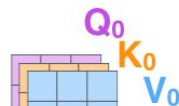
2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

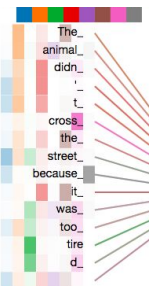


Z_0

W^O

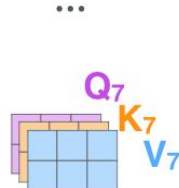
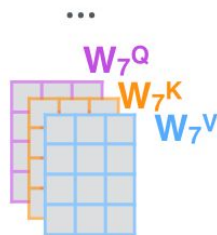
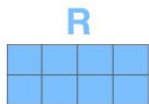


Z



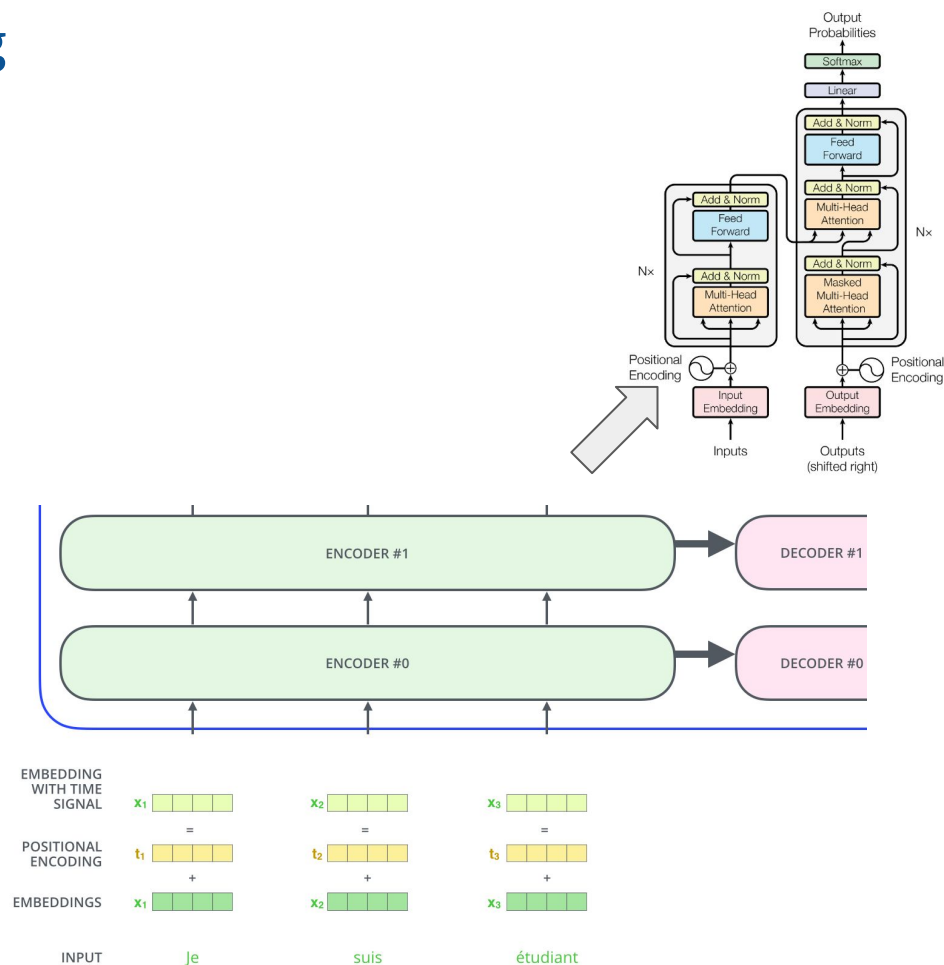
The_ animal_ didn_ ' t_ cross_ the_ street_ because_ it_ was_ too_ tire d_

* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



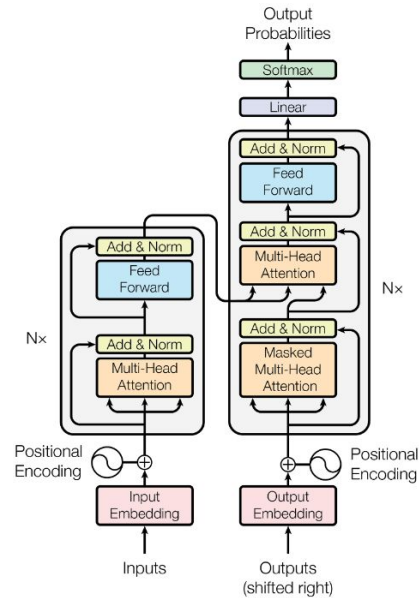
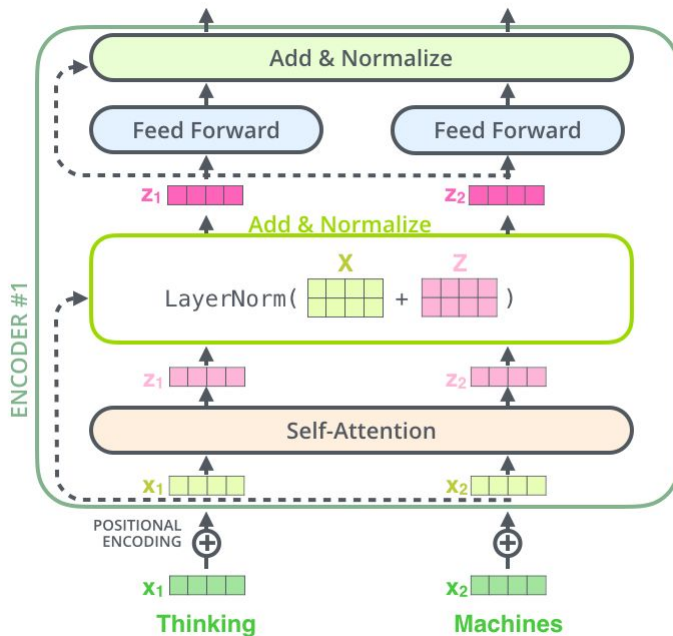
Transformers: Positional encoding

- Vectores que determinan el orden de las palabras en la oración.
- En general no se aprenden ya que la posición es bien conocida.



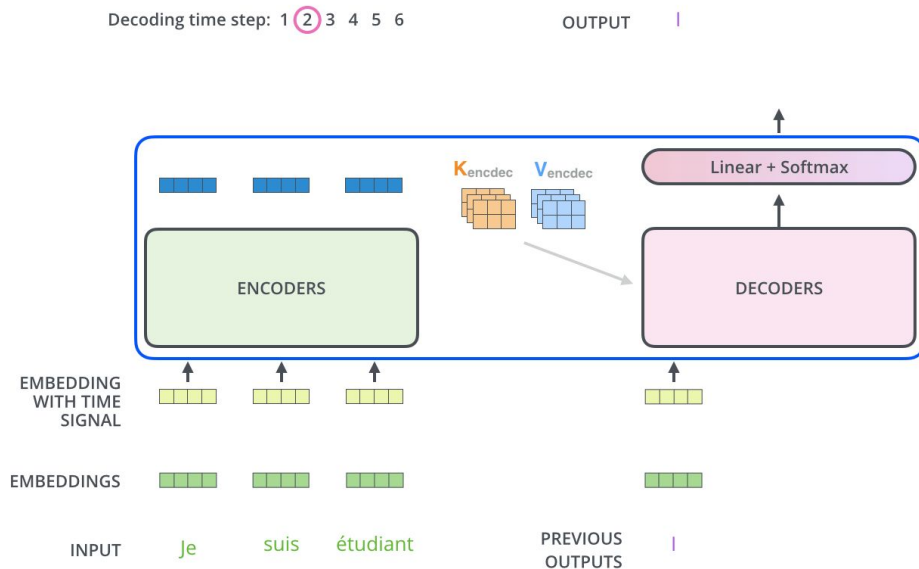
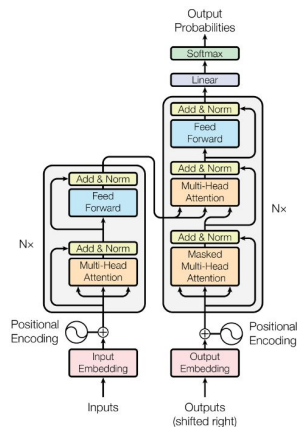
Transformers: Conexiones residuales

- Le permite a cada bloque saltarse capas.
- Evita el desvanecimiento del gradiente.
- LayerNorm: estabiliza el entrenamiento controlando el valor de las activaciones.



Transformers: del lado del Decoder

- Masked MHA: me permite entrenar con toda la secuencia de salida en una pasada. Enmascara los score antes del Softmax de manera que las salidas solo puedan atender a si mismas y las anteriores.
- Encoder-decoder attention: la salida del encoder se convierte en vectores de atención **K** y **V**, que se utilizan en dicha capa.



Transformers para series temporales



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Transformers para series temporales

- El éxito de los Transformers con datos secuenciales condujo naturalmente a su extensión hacia aplicaciones de series temporales.
- La utilización de transformer en general a sido para abordar el problema: **Forecasting de series temporales a largo plazo (LTSF)**.
- Se presentan variantes del Transformer para predicción de series temporales, destacando las limitaciones del modelo original, las mejoras introducidas, las razones de su mejor rendimiento y los desafíos que aún permanecen.

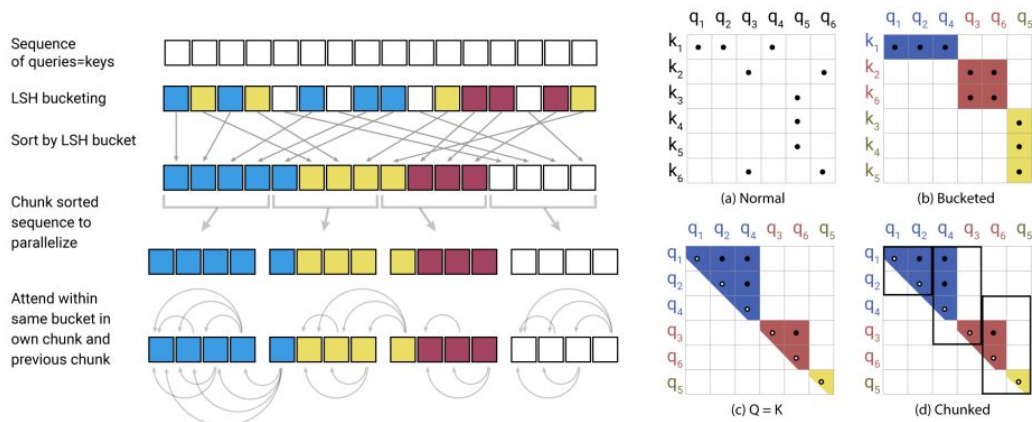
Transformers para series temporales: limitaciones del Transformer original

- Alto consumo de memoria para secuencias largas (L), ya que la atención requiere calcular y almacenar una matriz de tamaño $L \times L$ ($Q \cdot K^T$).
- Decoder autoregresivo, que genera las predicciones una a una, incrementando el tiempo de inferencia.
- Acumulación de errores, porque cada predicción depende de las anteriores y los errores se propagan a lo largo del horizonte de predicción.

Transformers: Variantes

Reformer (2020)

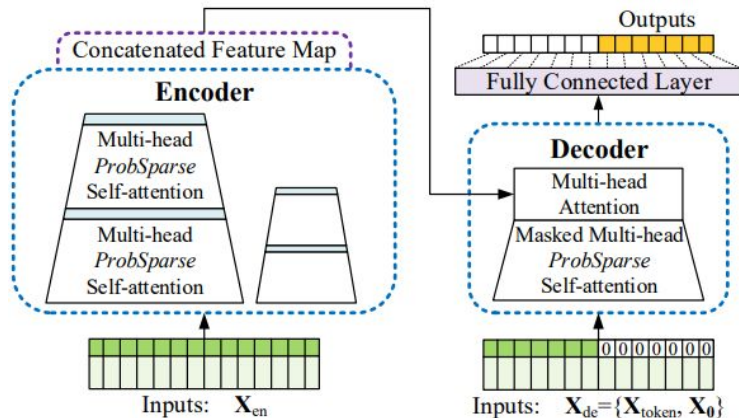
- Reduce la complejidad de la self-attention de $O(L^2)$ a $O(L \log(L))$.
- Utiliza **Locality-Sensitive Hashing (LSH)** para agrupar elementos similares y calcular la atención solo dentro de cada grupo.
- Permite procesar secuencias largas de forma más eficiente, manteniendo un rendimiento comparable al Transformer original.



Transformers: Variantes

Informer (2021)

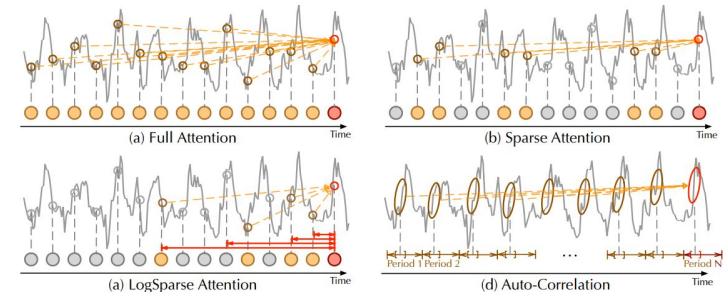
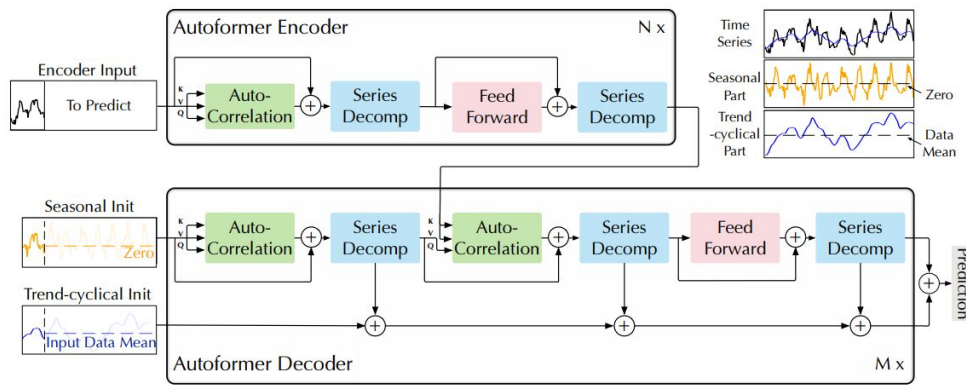
- Reduce la complejidad de la self-attention de $O(L^2)$ a $O(L \log(L))$ mediante **ProbSparse Self-Attention**.
- Compara las Queries con un vector uniforme:
 - si es similar -> poca información -> se descarta.
 - si la distribución es muy concentrada -> mucha información -> se mantiene.
- Utiliza un decoder generativo, que predice todo el horizonte de una sola vez y evita la generación autoregresiva.



Transformers: Variantes

Autoformer (2021)

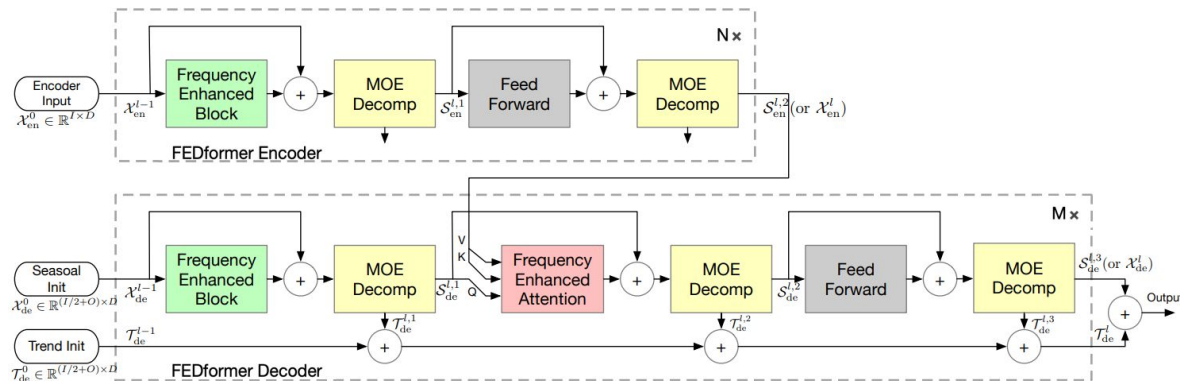
- Incorpora la descomposición de la serie temporal dentro del Transformer. Separa la serie en tendencia (trend) y estacionalidad (seasonality) para facilitar el aprendizaje.
- Reemplaza la self-attention por un mecanismo de Auto-Correlation, que identifica y agrega subseries con patrones periódicos similares.



Transformers: Variantes

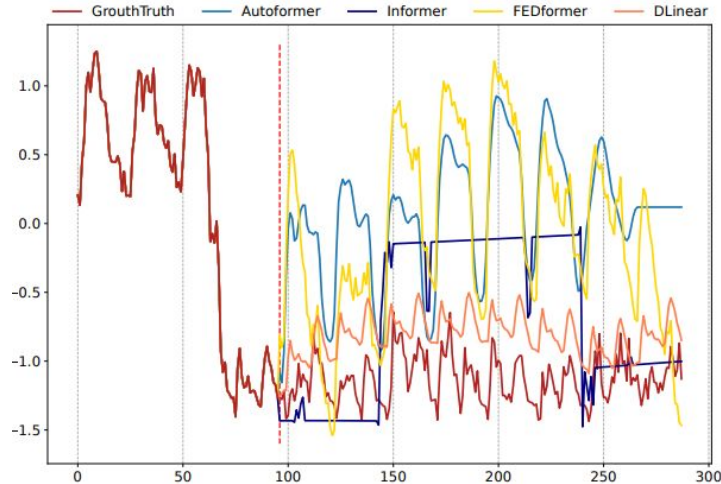
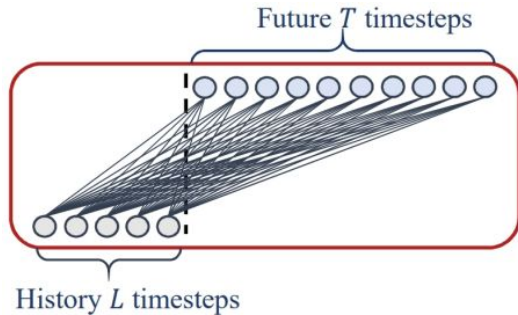
FEDformer (2022)

- Extiende Autoformer incorporando el análisis en el dominio de la frecuencia.
- Mantiene la descomposición en tendencia y estacionalidad.
- Utiliza una representación dispersa de frecuencias mediante la Transformada de Fourier o Wavelets.
- Modela únicamente las componentes de frecuencia más relevantes, reduciendo el costo computacional y mejorando la precisión en predicciones de largo plazo.



Transformers: Superando las limitaciones

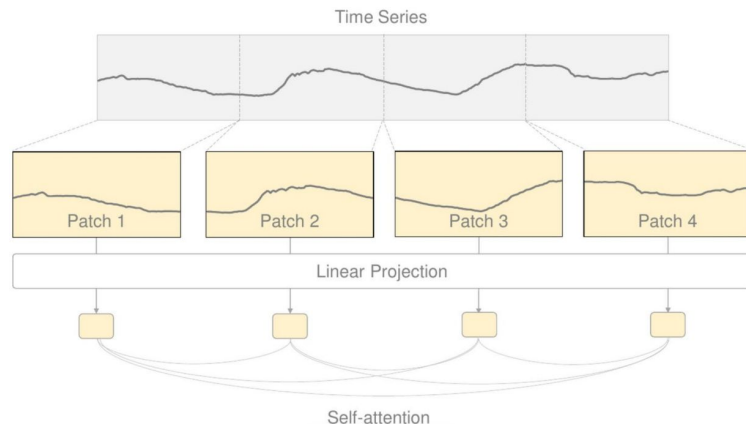
- Zeng, Ailing, et al. **"Are transformers effective for time series forecasting?."** Proceedings of the AAAI conference on artificial intelligence. Vol. 37. No. 9. 2023.
- El rendimiento superior de modelos lineales simples, como LTSF-Linear, frente a los modelos Transformer tradicionales ha puesto en duda la efectividad de los Transformers para la predicción de series temporales.



Patching en Transformers para Series Temporales

Idea principal

- A diferencia del NLP, donde cada token contiene información semántica rica, un punto individual de una serie temporal suele aportar poca información.
- El patching agrupa puntos consecutivos en segmentos (patches), permitiendo capturar contexto local, patrones temporales y reducir el costo de atención.
- Al procesar menos tokens, disminuye la complejidad de la self-attention y mejora la capacidad de modelar secuencias largas.



Patching en Transformers para Series Temporales

- **PatchTST**

- Divide la serie temporal en patches (segmentos consecutivos), inspirado en Vision Transformer (ViT).
- Utiliza únicamente el encoder del Transformer.
- **Idea clave:** Reemplaza puntos individuales por segmentos temporales para mejorar la representación de la serie.

- **MTST**

- Extiende PatchTST incorporando patches de múltiples escalas.
- **Idea clave:** Combina diferentes resoluciones temporales para capturar tanto detalles como tendencias globales.

- **PETformer**

- La salida del encoder en PatchTST aumenta significativamente el número de parámetros.
- Propone una técnica basada en placeholders para representar información pasada y futura en la misma escala temporal.
- Reduce más del 95% de los parámetros respecto a PatchTST.
- **Idea clave:** Reduce la complejidad del modelo manteniendo la información temporal entre pasado y futuro.

- Nie, Yuqi, et al. "A time series is worth 64 words: Long-term forecasting with transformers." arXiv preprint arXiv:2211.14730 (2022).
- Zhang Y, Ma L, Pal S, et al (2024e) Multi-resolution time-series transformer for long-term forecasting. In: International Conference on Artificial Intelligence and Statistics, PMLR, pp 4222–4230
- Lin S, Lin W, Wu W, et al (2024c) PETformer: Long-term time series forecasting via placeholder-enhanced transformer. <https://openreview.net/forum?id=u3RJbzzBZj>

Preguntas?



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY